Chat-Romanised Khmer to Khmer Script: Candidate Generation, Reverse Scoring and Contextual Decoding

Richard Cheam
richardcheam912@gmail.com

November 2026

Abstract

Khmer written in Latin letters in informal messages has no fixed spelling: one Khmer word has many romanisations, and one romanisation can stand for several Khmer words. We convert such messages to Khmer script with a small weighted noisy-channel decoder. A character-level encoder–decoder proposes Khmer candidates for each Latin word, a model of the reverse direction scores how well each candidate explains the typed letters, and a Khmer word-bigram model chooses one path through the candidates for the whole message. We report two findings. First, context and reverse scoring improve message conversion: on 165 held-out LLM-written chat messages, adding the reverse score lowered the character error rate from 10.56% to 9.37%, and on the messages used to fit the decoder, sentence context gave the largest observed reduction (11.4% to 4.4%). Second, with few human labels, filtered chat spellings generated by large language models taught the word model conventions that formal romanisation sources did not, raising word accuracy by +6.5 points on an early word test, with source and amount of data changing together. The message references are unreviewed LLM output and all human judgments come from one annotator, so the evidence concerns this system and these data rather than real multi-writer chat.

1 Introduction

Khmer has its own script, but informal Khmer is often written in Latin letters, a practice also described by the authors of the two existing conversion systems [6, 8]. This chat romanisation follows no standard and differs from the official romanisation systems: it drops most vowel distinctions, and writers keep their own habits, so *nis*, *nih* and *nes* can all be នេះ (‘this’). Search, text-to-speech and other software that expects Khmer script must first convert such text.

The task resembles the informal romanisation studied for South Asian languages [7, 11], with far fewer resources: we found no public corpus of real romanised Khmer chat with Khmer versions, and one public collection of word pairs, mostly dictionary-style [3]. Two systems appeared while this work was under way (§3); there is no shared benchmark on which to compare them with ours.

This report describes a small, CPU-only system and the evidence behind its design:

- a weighted noisy-channel decoder that combines a character-level word model, a reverse model scoring the typed letters, and a Khmer word-bigram model over paths that may join two Latin words (§4);
- an evaluation design for a setting with one native annotator and LLM-written messages, with the role and limits of each set made explicit (§5);
- two findings (§6): context and reverse scoring improve message conversion, and filtered LLM chat spellings help the word model; detailed comparisons are in the appendix.

2 The task and its ambiguity

The input is a short message in Latin letters, possibly mixed with Khmer script, digits and English words. The output is the message in Khmer script, with Khmer text, digits and punctuation left as they are. Usually one Latin word becomes one Khmer word; sometimes two neighbouring Latin words are one Khmer word (*pon man* → ប៉ុន្មាន, ‘how much’). Khmer does not separate words with spaces, so output spacing is not scored.

Table 1 shows the three sources of difficulty. One Khmer word has many spellings. One spelling can be several words, often common ones: *mok* is មក (‘come’) or មុខ (‘face, front’), and only the message tells them apart. And chat abbreviations (*hz* for ហើយ, ‘already’) cannot be read letter by letter. Output that looks like Khmer but is not a word is worse for a reader than leaving the word in Latin.

3 Related work

The Dakshina dataset [11] collected native-speaker romanisations of words and sentences in twelve South Asian languages, and Kirov et al. [7] showed that sentence context and a target-side language model help when romanisation is informal; they also built sentence training data by sampling word romanisations. Combining a candidate generator with a word-level language model follows earlier transliteration work [5].

Latin	Khmer (meaning)	Difficulty
knhom, *nhom*	ខ្ញុំ (I)	many spellings
nis, *nih*	នេះ (this)	many spellings
mok	មក (come), មុខ (face, front)	ambiguous
bong	បង (elder sibling), បង់ (pay)	ambiguous
mak	ម៉ាក់ (mum), ម៉ាក (brand)	ambiguous
hz	ហើយ (already)	abbreviation
pon man	ប៉ុន្មាន (how much)	two Latin words

Table 1: Chat spellings and what makes each hard.

Rescoring hypotheses with a model of the reverse direction is the noisy-channel approach of Yee et al. [15]; here it rescores word candidates inside the sentence search. For Khmer, AkaraAlpha [8] is a word-level recurrent model trained on the public pairs, and Akkhara [6] trains byte-level and small Transformer models on about 580k synthetic sentence pairs and reports results on 39 real chat sentences. We have not evaluated either on our data or ours on theirs. A companion report on English-to-Khmer phonetic spelling [1] uses the same model family.

4 Method

The design answers the difficulties of §2 in turn. Because one Khmer word has many romanisations, a model that generalises across spellings proposes candidates for each Latin word. Because one romanisation can be several words, the choice among candidates is left to the message context. Because the context model can prefer a frequent word that the typed letters hardly support, a second model checks each candidate against those letters. And because writers sometimes split one Khmer word with a space, the search also considers joined spellings. Figure 1 follows one message through the system: the message is split into Latin words, existing Khmer text, spaces and punctuation; the word model proposes 8 Khmer candidates for each Latin word and for each pair of neighbouring Latin words; the reverse model scores the typed letters given each candidate; and a search over the message picks the path with the highest total score, adding word-pair log-probabilities from a Khmer bigram model.

4.1 Character models

Both neural models are Transformer encoder–decoders [13] over characters. Figure 2 follows *mok* through the word model: letters become vectors, three encoder layers let every letter's vector draw on the whole word, and three decoder layers predict each Khmer character from the characters already written and from the encoder output. Figure 3 shows how candidates are generated, how the reverse model scores supplied letters instead, and how both are trained. The *word model* reads the letters of a Latin span x and writes a Khmer string y one character at a time, each step attending to the encoded input and to the characters already written:

$$p_\theta(y \mid x) = \prod_{t=1}^{|y|+1} p_\theta(y_t \mid y_{<t}, x), \tag{1}$$

where $y_{|y|+1}$ is the end mark. Beam search with width 8 gives the candidate set $C(x)$ with $\log p_\theta(y \mid x)$, the first score in Figure 1a. A character-level model is a practical starting point with about a hundred thousand word pairs: chat spellings are built from recurring letter patterns for Khmer syllables, so a model over characters can transfer them to spellings it has not seen, and a network of this size trains from scratch on a laptop (we follow the model family of our English-to-Khmer work [1]). Several candidates are kept because an ambiguous spelling has several plausible words and the word model alone cannot know which one the message needs. The candidates of each span are generated without looking at the rest of the message, so a word that is not among them cannot be chosen later; with 8 candidates this bound is measured in Appendix D. The *reverse model* has the same design but reads a direction tag followed by the Khmer candidate y, and its output is restricted to Latin letters and the end mark. It is never used to write a romanisation: the typed letters are supplied to its decoder as the sequence to score, and it returns the summed log-probability of each next letter and of the end mark (the second score in Figure 1a), that is, it scores the letters x that were actually typed (lower-cased, letters only; for a joined span, the two words concatenated) by teacher forcing,

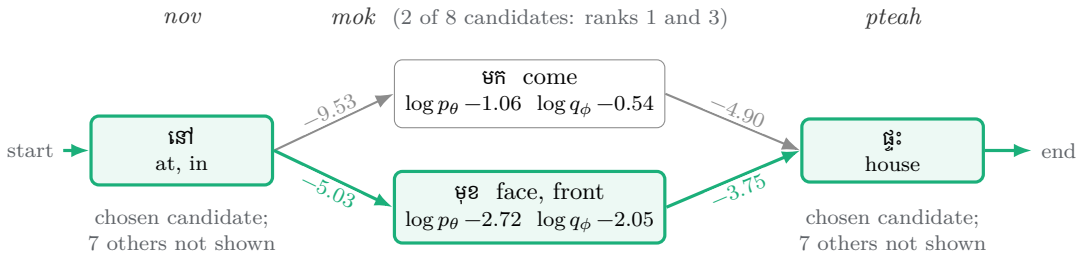
$$\log q_\phi(x \mid y) = \sum_{t=1}^{|x|+1} \log q_\phi(x_t \mid x_{<t}, y). \tag{2}$$

Training objectives. Both models minimise cross-entropy with label smoothing 0.1 [12]; for the reverse model the smoothing mass is spread over the allowed outputs only. The word model trains on (Latin spelling, Khmer word) pairs. On every eighth step it also trains a second decoder of the same design, reading the same encoder output, to predict the Khmer-adapted pronunciation (phone sequence) of Latin-script entries of a Khmer pronunciation lexicon [4], with loss weight 0.5; this decoder is discarded after training. The reverse model trains on (Khmer word, Latin spelling) pairs from the same training data without web-mined rows, drawing each Khmer word once per pass and one of its spellings at random. Table 2 gives the sizes and schedules of the two released networks.

4.2 Word-bigram model

Paired romanised messages are scarce, but Khmer-script text is plentiful, so the message context comes from a language model counted on unpaired Khmer

(a) message *nov mok pteah* (‘in front of the house’) → Latin words *nov*, *mok*, *pteah*; no existing Khmer text



path totals (candidate scores + all word-pair terms): នៅ មក ផ្ទះ −25.56 នៅ មុខ ផ្ទះ **−23.08** ⇒ output **នៅមុខផ្ទះ**

(b) Reverse score changes the choice. *mak tek chol pteah ot* (‘Mum, did water get into the house?’)

candidate	log p_θ	log q_ϕ	total, γ=0	total, γ=1
ម៉ាក់ mum	−2.01	−1.94	−55.73	**−57.67**
ម៉ាក brand	−0.97	−6.09	**−53.27**	−59.37

(c) Joined span. *pon man* (‘how much’)

path	total
ប៉ុន្មាន (one span, cost $\beta = 8$ included)	**−21.91**
ប៉ុន + ម៉ាន់ (two spans)	−32.04

Figure 1: Worked examples from the released system; all scores are natural logarithms from a diagnostic run ($\alpha = 1$, $\gamma = 1$, $\beta = 8$). (a) The candidate lattice of *nov mok pteah*. Edge labels are word-pair log-probabilities $\log P(\text{next} \mid \text{previous})$; the chosen path is green. Both the word model and the reverse model prefer មក ‘come’, but the pairs នៅ មុខ and មុខ ផ្ទះ are much more probable, so the path through មុខ has the higher total. Only *mok* is expanded; the other words take part through their chosen candidates and the pair terms, and the totals also include the start and end terms. (b) The word model prefers ម៉ាក ‘brand’; the typed *mak* is much less likely given that candidate ($\log q_\phi$), which reverses the choice. (c) The joined spelling wins over the best two-word path despite the join cost.

	Word model	Reverse model
Input	26 letters	tag + Khmer[a]
Output	81 Khmer chars	26 letters[a]
Width / FF	192 / 512	192 / 512
Layers	3 + 3	3 + 3
Heads	4	4
Parameters	2.58M	2.61M
Training	40 epochs[b]	31,265 steps
Checkpoint	validation	held-out NLL
Use	beam 8	scoring

Table 2: The two networks of the released system. Both: pre-layer normalisation, learned character and position embeddings, GELU, dropout 0.1, AdamW (learning rate 10^{-3}, weight decay 0.01), 500 warm-up steps then cosine decay, gradient clipping 1.0, batches of 128. [a]The reverse model shares one character table of 109 entries (two direction tags, 26 letters, 81 Khmer characters) and masks its outputs to letters and the end mark. [b]116,921 word pairs per epoch; the checkpoint with the best mean greedy exact match on three validation sets is kept. The reverse checkpoint has the lowest negative log-likelihood of the real spellings of held-out Khmer words.

text. A word-pair model can tell មុខ ‘front’ from មក ‘come’ after នៅ ‘at’ (Figure 1a), but it sees only the previous word: a choice that depends on a word further back, such as ព្រុស ‘bark’ after an earlier mention of a dog, is outside its reach. We chose a bigram model for its simplicity and size (21 MB of counts) and did not compare higher orders.

Word and word-pair counts come from 200,000 Khmer web documents [10], split into sentences at punctuation. Khmer text is split into words by greedy longest match against a vocabulary of 92,905 words (a lexicon’s headwords plus every training target); pairs seen fewer than twice are not stored. Raw relative frequencies would give every pair absent from the table probability zero, and its log score would be undefined, although many valid word pairs never occur in any sample of text. We therefore use absolute discounting [9], one of the standard smoothing methods compared by Chen and Goodman [2]: a fixed amount d is subtracted from each stored pair count and the freed probability is given to all words in proportion to a unigram distribution,

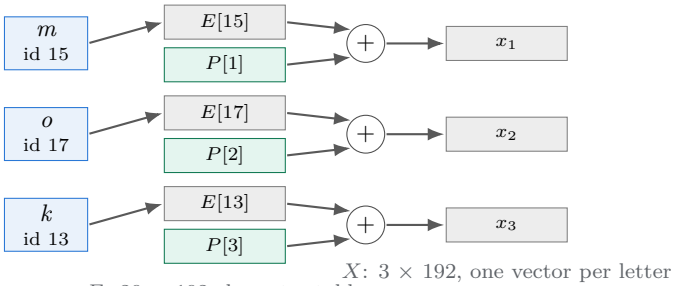
$$P(w \mid v) = \frac{\max(c(v,w) - d,\, 0)}{c(v)} + \lambda(v)\, P_1(w), \quad (3)$$

where $c(v)$ counts all occurrences of v and $c(v,w)$ the stored pair counts. The back-off weight $\lambda(v)$ is the discounted mass plus the mass of the pairs that were not stored, so the probabilities after v sum to about one; P_1 is an add-one unigram distribution (Appendix A). Sentence start is a predecessor and sentence end a predicted word; a predecessor never seen uses P_1 alone, and a word outside the vocabulary receives a fixed floor of $\log 10^{-8}$. The discount is fixed at $d = 0.75$ and was not tuned; Ney et al. [9] derive d from count statistics, which we did not use. Absolute discounting is our choice of smoothing, not the result of a comparison.

4.3 Decoding over paths

A message is read as a sequence of units: Latin words, existing Khmer text, spaces, and other characters such as punctuation and digits. Existing Khmer text is a continuous sequence of Khmer-script characters,

(a) From letters to vectors: embedding lookup plus learned position



X: 3 × 192, one vector per letter

E: 29 × 192 character table

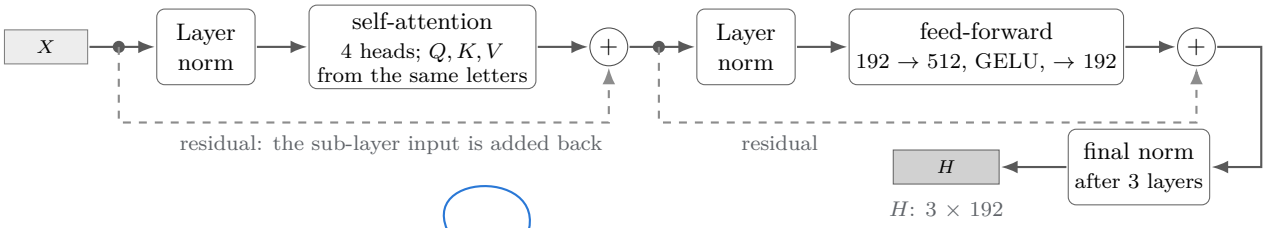
P: 48 × 192 position table

first 4 of 192 values, letter m at position 1

$E[15]$	−0.23	0.50	−0.40	0.75	…
$+ P[1]$	−1.91	−0.14	−0.24	0.61	…
$= x_1$	−2.14	0.36	−0.64	1.36	…

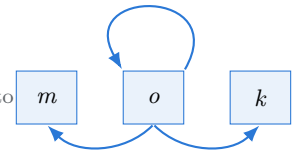
Each letter's id selects a row of the character table E; the row of the position table P for its place in the word is added, number by number. Both tables are learned. Similar-sounding letters do not start with similar rows; such regularities are built up by the encoder layers. Ids 0, 1, 2 are padding, start and end.

(b) One encoder layer, repeated 3 times: each letter's vector is updated using all input letters



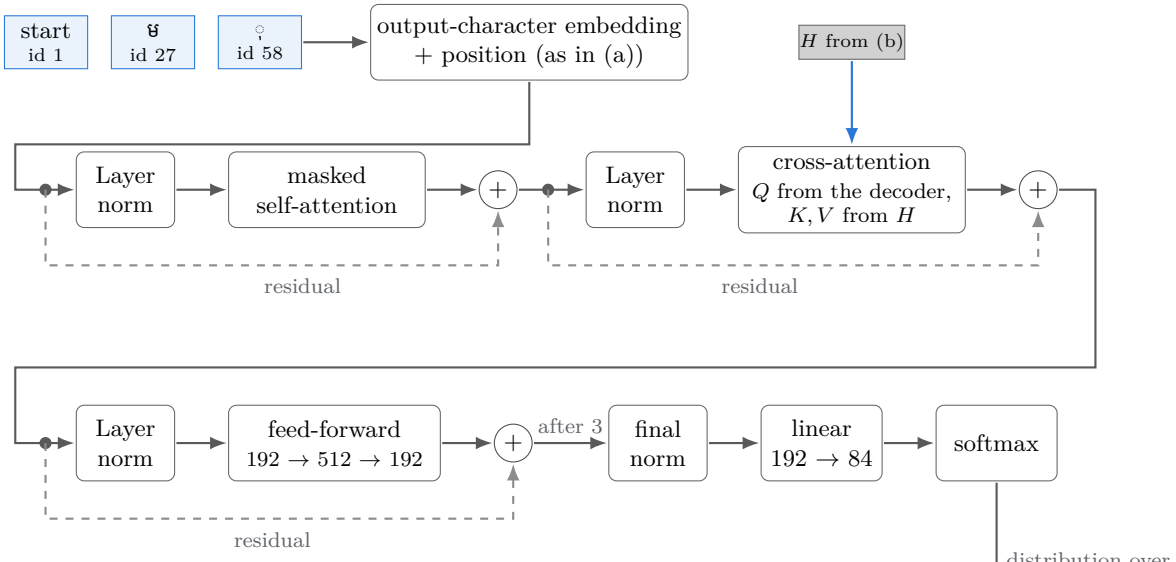
H: 3 × 192

example: the vector at o attends to



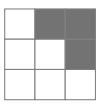
all three letters, so its new vector reflects the whole word. H, one vector per letter, supplies the keys and values of the decoder's cross-attention in (c).

(c) One decoder step, layers repeated 3 times: given start + ʁ + ុ, predict the next character

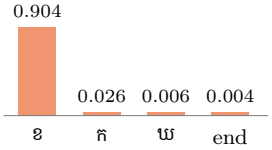


distribution over 84 outputs

(d) Output of this step, and the causal mask used by masked self-attention



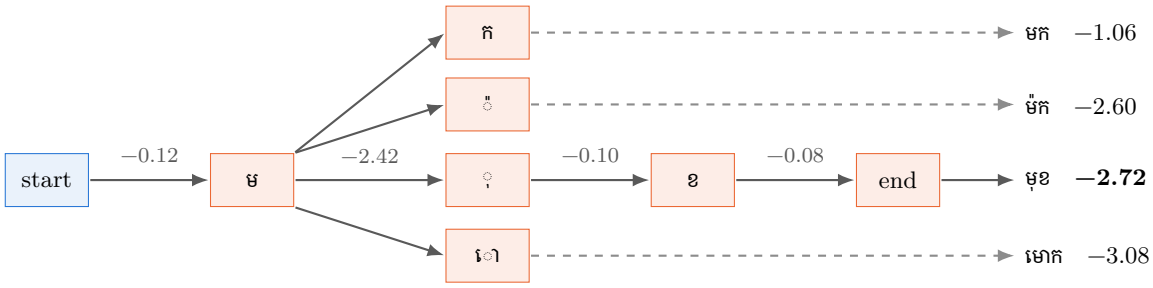
row i = position i of the prefix; it may attend only to positions $\leq i$ (dark = blocked), so no position sees characters not yet written



measured probabilities of the next character, top 4 of 84 outputs

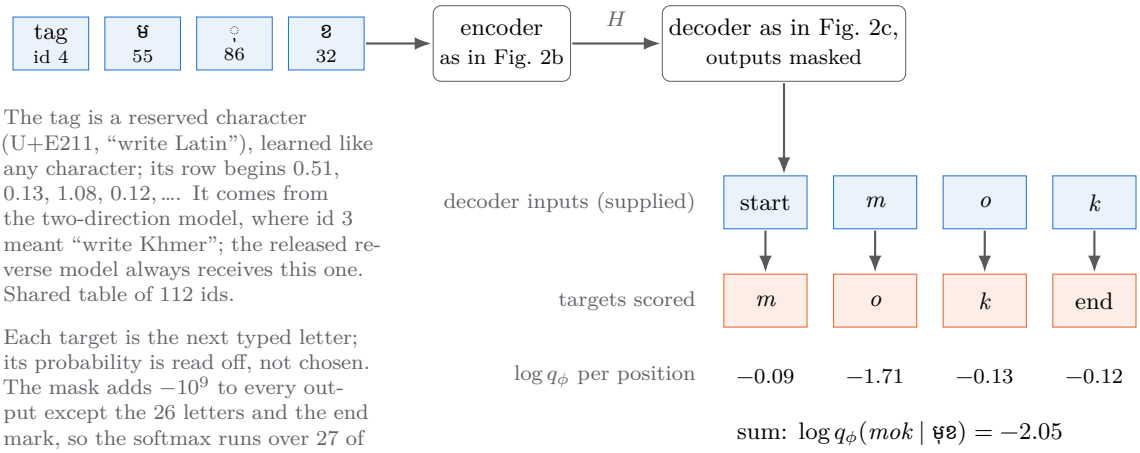
Figure 2: Inside the word model, for *mok* → ម្ʁ, as implemented. Pre-layer normalisation: each sub-layer normalises its input, and its output is added to the unnormalised input at the marked branch points (dots). (a) Letters become vectors. (b) One of three encoder layers; the final normalisation gives H. (c) One of three decoder layers at the step after start + ʁ + ុ, followed by the final normalisation, a projection to the 84 output ids (81 Khmer characters, padding, start, end) and a softmax. (d) The resulting measured distribution and the causal mask. Dropout (0.1) is used only in training; other settings are in Table 2.

(a) Generation: the word model writes candidates one character at a time (beam search, width 8)



Solid arrows: steps drawn in full, with the next-character log-probabilities of the path to မុខ; its score −2.72 is their sum, including the end mark. Dashed arrows: the remaining steps of the other candidates are omitted. These are 4 of the 8 completed candidates for *mok*; all 8 with their scores form $C(mok)$ in Figure 1a.

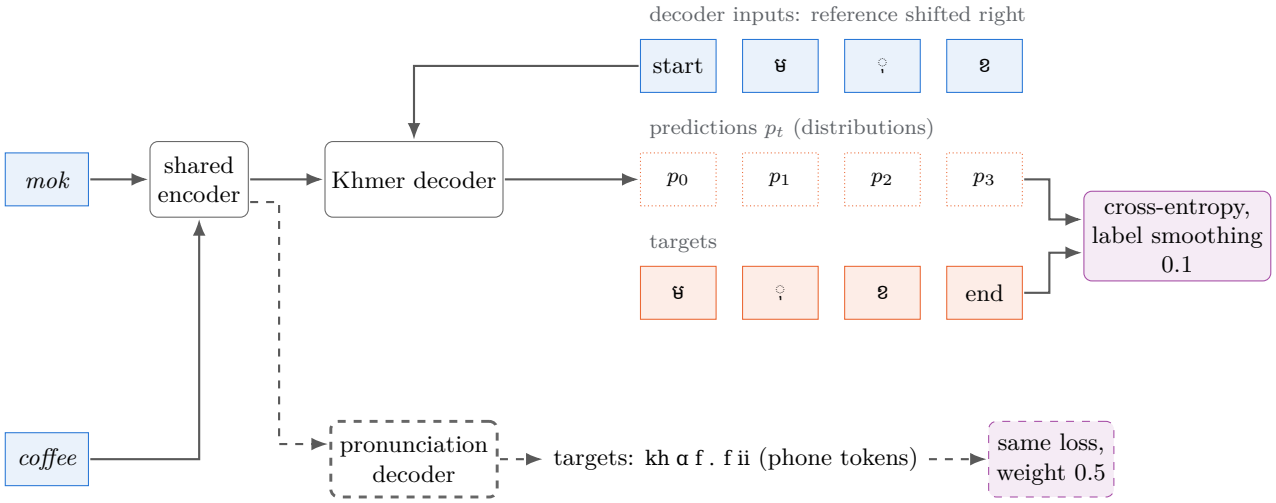
(b) Reverse scoring: the typed Latin is supplied and scored given a Khmer candidate (not generated)



The tag is a reserved character (U+E211, "write Latin"), learned like any character; its row begins 0.51, 0.13, 1.08, 0.12, …. It comes from the two-direction model, where id 3 meant "write Khmer"; the released reverse model always receives this one. Shared table of 112 ids.

Each target is the next typed letter; its probability is read off, not chosen. The mask adds -10^9 to every output except the 26 letters and the end mark, so the softmax runs over 27 of 112 outputs. Same architecture as Fig. 2, separate weights.

(c) Training: predictions are compared with the reference shifted by one position



p_t: the decoder's distribution over the 84 outputs at position t; because the inputs are the reference shifted right by one, each prediction sees only earlier characters. Training only (dashed borders and arrows): on every 8th step a batch of Latin-script lexicon entries goes through the shared encoder and the pronunciation decoder, and the step's loss is the Khmer loss plus 0.5 times the pronunciation loss. Inference uses only the encoder and the Khmer decoder.

Figure 3: Generation, reverse scoring and training, for *mok* and မុខ. (a) Beam search keeps the 8 best prefixes at each step and ranks completed candidates by their summed log-probabilities (no length normalisation). (b) The reverse model receives the candidate after its direction tag, and its decoder is given the typed letters shifted by one position, so each position scores the next typed letter; the sum is the reverse score of Eq. (2). (c) Training uses the same shift on the reference Khmer; the pronunciation decoder has the design of the Khmer decoder, its own embeddings and output layer, and is discarded after training. Numbers are measured with the released models.

which may hold several words. In ថ្ងៃនេះ *bong mok* ផ្ទះ ('today, [elder sibling] comes home'), the system keeps ថ្ងៃនេះ and ផ្ទះ as typed, converts *bong* and *mok*, and uses the existing Khmer text as context when choosing their spellings; the output is ថ្ងៃនេះ បងមក ផ្ទះ. For scoring, the bigram model splits existing Khmer text into its vocabulary words (ខ្ញុំទៅផ្ទះ becomes ខ្ញុំ | ទៅ | ផ្ទះ).

A *span* is the Latin input that receives one Khmer candidate: a single Latin word, or two Latin words separated by exactly one space (a join). Writers sometimes put a space inside one Khmer word, typing its syllables separately (*pon man* for ប៉ុន្មាន), so the decoder considers both readings rather than trusting the space. Each span e with letters x_e has a *candidate set* $C(x_e)$, the word model's 8 best Khmer spellings. Laid out from left to right, the candidate sets form a *lattice*, and a *path* π chooses one candidate y_e for each span from the start of the message to its end (Figure 1a: three spans, one path in green). Punctuation ends a sentence. Splitting each chosen candidate and the existing Khmer text into vocabulary words gives, per sentence s, a word sequence $w_1^s \dots w_{m_s}^s$. The path score is a weighted sum of log-probabilities minus a penalty for joins:

$$S(\pi) = \sum_{e \in \pi} \alpha \big[\log p_\theta(y_e \mid x_e) + \gamma \log q_\phi(x_e \mid y_e) \big] + \sum_s \sum_{k=1}^{m_s+1} \log P(w_k^s \mid w_{k-1}^s) \; - \; \beta \, J(\pi), \tag{4}$$

where $J(\pi)$ counts joins, w_0^s is sentence start and $w_{m_s+1}^s$ sentence end. In Figure 1a the candidate boxes show the two terms of the first sum, the edge labels are bigram terms and the path totals are $S(\pi)$; the joined path of Figure 1c includes $-\beta$. The join cost β keeps two separately typed words apart unless the joined spelling is clearly better; its value was selected on development messages and need not suit other writers or domains.

Because the bigram term depends only on the previous Khmer word, the part of a path that matters for its future is its position in the message and its last Khmer word. Dynamic programming in the manner of Viterbi [14] therefore keeps, for each position in the message and last word, only the best partial path, and returns $\arg\max_\pi S(\pi)$ exactly over the lattice (recurrence in Appendix A), with work that grows linearly with message length rather than with the number of paths. The search is exact only within the generated candidate sets. There is no length normalisation. We call this search *contextual decoding*, in contrast to converting each word on its own.

Why the reverse score. The word model scores how well a candidate follows from the letters, and the bigram model rewards frequent words and pairs; neither asks directly whether the candidate would explain all of the typed letters. A frequent candidate can therefore win although writers rarely type it this way. The reverse score supplies that missing evidence: $\log q_\phi(x \mid y)$ is high when x is a typical romanisation of y. In Figure 1b the word model prefers ម៉ាក 'brand' for *mak*, but *mak* is a much more typical spelling of ម៉ាក់ 'mum' ($\log q_\phi$ -1.94 against -6.09), which reverses the choice. The same evidence penalises a correct word typed in an unusual way (§6.1). Following the noisy-channel approach [15], the decoder is a weighted combination: p_θ and q_ϕ measure the fit between letters and candidate, the bigram term the fit to the neighbours, and α, γ, β weight them. The reverse term lowers unsupported candidates but does not guarantee that they lose.

Source	Rows	Kind
Public romanised pairs [3]	24,016	external
Web-mined pairs	34,345	automatic
LLM chat spellings	48,057	synthetic
Curated loanword list	1,480	reviewed
Annotator typing	408	human
Checked LLM suggestions	580	human-checked
Annotator examples	143	human
Common words from the public validation split	59	external

Table 3: Training pairs of the released word model, after filtering. The four reviewed or human sources are repeated four times per epoch (116,921 pairs per epoch). Web-mined pairs come from bracketed Latin spellings in Khmer web text [10] and Wikipedia; synthetic spellings were filtered automatically, never checked by hand. The origin of the public pairs is undocumented. The public pairs come with a validation split; its pairs whose Khmer word is among the 500 most frequent words of the web text were moved into training (last row), so that frequent words are not missing from training, and the rest of that split stayed held out for checkpoint selection.

What is learned, counted, fixed or selected. The network weights θ and ϕ are learned from word pairs, with checkpoints chosen on validation word sets. The bigram probabilities are counted from Khmer web text. The discount $d = 0.75$, the bigram order, the minimum pair count of 2, the beam of 8 and the absence of length normalisation are fixed design choices. The weights $\alpha = 1.0$ (from $\{0.5, 1, 2, 3\}$) and $\beta = 8.0$ (from no joins or $\{0, 2, 4, 8\}$, the largest value tried) were selected on the fit split of Messages A, and $\gamma = 1$ (from $\{0.25, 0.5, 1, 2\}$) on the tuning half of Messages B (§5); $\gamma = 0$ removes the reverse model.

5 Data and evaluation design

5.1 Training data

Table 3 lists the training data of the released word model. Over the course of the study the pool also contained a rule-romanised pronunciation lexicon (removed early, Appendix G), spellings generated by a third LLM provider, and LLM drafts whose generating model was not recorded; the last two were removed for the release (Appendix E).

LLM spellings. LLMs were asked how Khmer speakers would type each of the most common Khmer words in chat, giving several spellings per word. No spelling was checked by hand. A fixed filter dropped spellings that were not plain *a*–*z*, were much longer than the Khmer word, conflicted with the annotator's spellings, or were given for several words (then kept only for the most common one). The models often disagreed, and on the annotator's training words the first suggestion matched one of the annotator's spellings for only about a third of the words, so these are noisy labels.

5.2 Held-out data

Held-out items are removed from training by a filter. Two policies were used, and they test different things. *Target exclusion* removes every training row whose Khmer word is a test word, so the test measures words the model has never produced. *Pair exclusion* removes only the exact (spelling, word) pairs, so a test word may occur in training with other spellings and the test measures new spellings, mostly of known words. The early data comparisons used target exclusion for the first word test and the public regression tests. Once contextual decoding was introduced, the fresh test and the romanised validation set used pair exclusion, the public regression tests stopped blocking training and were scored only on items whose target is absent from training, and the English evaluation panels kept target exclusion. Results under different policies are not directly comparable. For the released model, 70 of the 148 fresh-test words occur in training with other spellings and 78 do not.

5.3 Evaluation sets and measures

Table 4 summarises every evaluation set: what it holds out, which choices used it, and what its score measures.

Word accuracy. A spelling is correct if the top output is exactly the intended Khmer word. Word accuracy is a macro average over Khmer words: each word scores the share of its spellings written correctly, and words count equally, so a word typed three ways does not weigh more than a word typed once. In the data comparisons the word model is scored alone (beam 4, preferring a lexicon word among the top candidates); the released system's figures convert each spelling through the full pipeline. These two settings are not comparable.

Character error rate. CER is the Levenshtein distance between output and reference divided by the reference length, pooled over messages. From the corrected contextual decoding on, spaces, punctuation and digits are removed before scoring; the first contextual-decoding comparison removed only spaces. References in both message sets are the generating LLM's own Khmer versions, unreviewed.

Intervals. Word-test intervals resample words; message intervals for Messages B resample whole generation calls, because messages from one call share topics and style, and every system is scored on the same resample. Where several seeds exist the per-seed differences are reported; the intervals cover the sampling of items for fixed models, not retraining. Decision rules were fixed before the corresponding results were seen.

Blinded comparisons. Messages are drawn by a hash from those on which two systems write different Khmer, and shown with the two outputs in random order, without references or system names. The judge marks one side better, about the same, or both bad. Each preference is conditional on that sample of differing messages and on one judge.

6 Findings

Two findings carry the paper: scoring candidates in context, and in both directions, improved message conversion (§6.1); and filtered LLM chat spellings taught the word model conventions that formal sources did not (§6.2). Section 6.3 states what limits both. Data mixtures, the retraining of the released models, baseline exposure, candidate coverage, seed values and learning curves are in Appendices C–G.

6.1 Context and reverse scoring improve message conversion

The reverse score, on held-out messages. On the 165 test messages of Messages B (from 11 generation calls), read once after γ and every other setting had been fixed on the tuning half, adding the reverse score ($\gamma = 1$ against $\gamma = 0$, same word model and decoder) lowered the character error rate from 10.56% to 9.37%: -1.19 points $[-1.84, -0.51]$, with the interval resampling generation calls, and all 3 paired retrainings moving the same way. This is the paper's strongest evidence, because these messages took no part in any choice.

What the reverse score adds is shown in Figure 1b. For *mak*, the word model scores ម៉ាក 'brand' higher, and without the reverse score that choice stands; the reverse model finds the typed *mak* much more likely for ម៉ាក់ 'mum' ($\log q_\phi$ -1.94 against -6.09), which reverses the choice. The same evidence can hurt a correct word typed unusually: in *knhom lieng mok* ('I wash my face'), *lieng* is a rare spelling of លាង 'wash' ($\log q_\phi = -5.50$ against -0.63 for លៀង), and the output becomes ខ្ញុំលៀងមក instead of ខ្ញុំលាងមុខ.

The blinded comparison points the same way, with a narrower basis. One judge, the author, compared the two systems' outputs on 40 of the 75 test messages where they differed, without references or system labels: the version with the reverse score was better in 17 messages and the version without it in 7, with 5 judged the same and 11 both bad (one-sided sign test $p = 0.032$). This says nothing about the messages

Set	Items	Held out	Used for	Score
Validation words	release: 1,576 romanised, 320 names, 179 loans	exact pairs	checkpoint selection	greedy exact match
First word test	160 common words, 209 spellings (annotator)	target exclusion	early data comparisons; later retired into training	word accuracy (macro)
Fresh word test	148 less common words, 179 spellings (annotator); 225 sound-alike variants of 116 words	exact pairs and variants	word-level comparisons; checks for regressions	word accuracy (macro)
Public regression tests	romanised test (2226 items); names (223)	target exclusion; later scored only on items with targets absent from training (1169; 38)	checks that changes do not lose accuracy on public data	item accuracy
Messages A	447 LLM-written chat messages with LLM Khmer	hash split: 279 fit, 168 check	fit: α, β; check: later comparisons	CER, pooled
Messages B	450 LLM-written messages from 30 generation calls	split by call: 285 tune (19 calls), 165 test (11 calls)	tune: γ and reverse model; test: reverse-score decision, read once; later all 450 for the release comparison and seed choice	CER, pooled; bootstrap over calls
Blinded comparisons	4 comparisons; 162 judgments of 157 distinct LLM-written messages	sampled from messages where two systems differ	decisions	preference counts

Table 4: Evaluation sets. All human items come from one annotator, a native Khmer speaker who is also the author.

where the outputs agree. On the fresh word test, with each spelling converted alone, accuracy moved from 56.8% to 61.7% (+4.9 points [−0.3, +10.4]). The cost is 0.49 s of reverse scoring per message on a laptop CPU (165 messages, no cache).

Sentence context. The largest observed reduction in message error came from decoding the whole message with the word-bigram model instead of taking each word's top candidate. On the 279 fit messages of Messages A, the CER fell from 11.4% to 4.4% for our word model, and from 16.1% to 8.9% for the baseline's. These messages were also used to fit the decoder's weights α and β, so the size of the reduction is optimistic; the comparison holds the word model fixed and so isolates the decoder. Appendix D shows how far the choice falls short of the candidates, and Appendix G gives the blinded comparison of contextual decoding with the baseline.

6.2 Filtered chat spellings helped the word model

Formal romanisation and chat typing differ. In chat, *tha* is the common spelling of ថា 'that'; a model trained on the public dictionary-style pairs, a rule-romanised lexicon and the annotator's training words read it literally, as តា. Adding about 12.6k spellings of common words, generated by one LLM and filtered automatically, raised macro word accuracy on the first word test by +6.5 points [+3.1, +10.1] (three seeds each, both models with every test word excluded from training), the first change in the series whose interval excludes zero. The added rows changed both the source and the amount of data, so the result shows that adding filtered LLM spellings helped these models on this test, not that the source matters more than the quantity, nor that unchecked LLM labels capture human conventions in general. Appendix G gives the full series of training mixtures, including spellings from three LLMs.

6.3 What limits these conclusions

Three features of the evidence bound both findings. The message sets were written by an LLM and their references are its own unreviewed Khmer, so lower error means closer agreement with that LLM on its own chat, not established quality on chat from many writers. Every human judgment comes from one annotator who is also the author; blinding removes system labels, but the judgments are not independent review. Word-level comparisons depend on what was held out: against the baseline, which had trained on most words of the first test, the ranking reversed when a fresh test was used, and excluding test words from training also removed the commonest chat words (Appendix C). Retraining the same recipe moved results by a few points (Appendix F), so single-seed differences of that size were not reliable.

7 Limitations

- **One annotator.** The typed test words, the checked suggestions and all 162 blinded judgments come from one native speaker, who is also the author. Other writers appear only through the sound-alike variants.

- **No real multi-writer chat.** Message sets and blinded comparisons use LLM-written chat; references are unreviewed LLM output. Agreement with them, or a preference among differing outputs, does not establish quality on real chat.
- **Selection effects.** The released seed and γ were chosen on Messages B, and Messages A were reused across several comparisons.
- **Exact match.** Exact agreement undercounts acceptable spellings and treats near misses and non-words alike.
- **Non-words and abbreviations.** The system can output Khmer strings that are not words; abbreviations standing for whole words or names (*fb*, *BKK*) and English words meant to be written out in Khmer remain frequent errors.
- **Data provenance.** The public pairs' origin is undocumented, and LLM-generated spellings are subject to their providers' terms.

8 Conclusion

A word model, a reverse model scoring the typed letters and a Khmer word-bigram model, combined as a weighted noisy-channel decoder over paths that may join words, convert chat-romanised Khmer with small CPU models. Two findings stand out. Scoring candidates in context and in both directions improved message conversion: sentence context gave the largest observed reduction in message error, on messages that also fitted the decoder, and the reverse score gave a further reduction on held-out messages that one judge preferred where the outputs differed. Filtered LLM spellings of common words taught the word model chat conventions that formal sources did not, a gain in which source and amount changed together. Both findings rest on LLM-written messages and one annotator, and the word-level comparisons proved sensitive to what was held out and to retraining.

Open questions are how well these results hold on real chat from many writers, whether message-level training can resolve choices that word-pair context misses (ព្រុស 'bark' against the far more common ព្រោះ 'because' after a dog), how to handle abbreviations that stand for whole words or names, and how this system compares with other Khmer systems on a shared benchmark.

References

[1] Richard Cheam. en-km-phonetic-transliterator (model repository, tag v0.2.0). `https://huggingface.co/ric912/en-km-phonetic-transliterator`, 2026. Weights CC BY-NC-SA 4.0.

[2] Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech & Language*, 13(4):359–394, 1999. doi: 10.1006/csla.1999.0128.

[3] Chhunneng. khmer-text-transliteration. `https://github.com/Chhunneng/khmer-text-transliteration`. Commit fe8028f1; MIT licence declared; the origin of the pairs is not documented. Accessed 2026-11-01.

[4] Google. Khmer pronunciation lexicon (`km/data/lexicon.tsv`), `google/language-resources`. `https://github.com/google/language-resources`. Commit bc72dd65 (2018-06-21); CC BY 4.0, Copyright 2018 Google Inc.

[5] Roman Grundkiewicz and Kenneth Heafield. Neural machine translation techniques for named entity transliteration. In *Proceedings of the Seventh Named Entities Workshop*, pages 89–94, Melbourne, Australia, 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-2413. URL `https://aclanthology.org/W18-2413/`.

[6] Muyleang Ing. Akkhara: Sing Khmer to Khmer script transliteration models. Hugging Face model card, `https://huggingface.co/INGMUYLEANG/akkhara-byt5-small`, 2026.

[7] Christo Kirov, Cibu Johny, Anna Katanova, Alexander Gutkin, and Brian Roark. Context-aware transliteration of Romanized South Asian languages. *Computational Linguistics*, 50(2):475–534, 2024. doi: 10.1162/coli_a_00510. URL `https://aclanthology.org/2024.cl-2.2/`.

[8] Netra AI Lab. AkaraAlpha: An efficient romanized Khmer–Khmer script transliteration model. Technical report and code, `https://github.com/netra-ai-lab/Romanized-Khmer-to-Khmer-Transliteration`, 2026.

[9] Hermann Ney, Ute Essen, and Reinhard Kneser. On structuring probabilistic dependences in stochastic language modelling. *Computer Speech & Language*, 8 (1):1–38, 1994. doi: 10.1006/csla.1994.1001.

[10] Guilherme Penedo, Hynek Kydlíček, Vinko Sabolčec, Bettina Messmer, Negar Foroutan, Amir Hossein Kargaran, Colin Raffel, Martin Jaggi, Leandro Von Werra, and Thomas Wolf. FineWeb2: One pipeline to scale them all – adapting pre-training data processing to every language, 2025. URL `https://arxiv.org/abs/2506.20920`.

[11] Brian Roark, Lawrence Wolf-Sonkin, Christo Kirov, Sabrina J. Mielke, Cibu Johny, Isin Demirsahin, and Keith Hall. Processing South Asian languages written in the Latin script: the Dakshina dataset. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 2413–2423, Marseille, France, 2020. European Language Resources Association. URL `https://aclanthology.org/2020.lrec-1.294/`.

[12] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2818–2826, 2016. URL `https://arxiv.org/abs/1512.00567`.

[13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL `https://arxiv.org/abs/1706.03762`.

[14] Andrew J. Viterbi. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2):260–269, 1967. doi: 10.1109/TIT.1967.1054010.

[15] Kyra Yee, Yann Dauphin, and Michael Auli. Simple and effective noisy channel modeling for neural machine translation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 5696–5701, 2019. URL `https://aclanthology.org/D19-1571/`.

A Decoding details

Bigram probabilities. Let $c(v)$ be the corpus count of v, which equals the number of pairs starting with v (each occurrence is followed by a word or by sentence end), $c(v, w)$ the stored count of the pair (pairs seen at least twice are stored), $N_+(v)$ the number of stored pairs starting with v and $K(v) = \sum_w c(v, w)$ their total. Then

$$\lambda(v) = \frac{c(v) - K(v) + d\,N_+(v)}{c(v)}, \qquad P_1(w) = \frac{c(w) + 1}{T + |V| + 1},$$

so the back-off mass is the discounted amount d per stored pair plus all counts of pairs that were not stored. T is the number of word tokens plus one sentence end per sentence, and $|V|$ the number of vocabulary entries. Sentence end is predicted like a word, with a count equal to the number of sentences; sentence start is only a predecessor. A predecessor with $c(v) = 0$ uses $P_1(w)$ alone. A word outside the vocabulary receives a fixed $\log P = \log 10^{-8}$, a floor outside the normalised distribution. The discount is $d = 0.75$.

Recurrence. The message units $t_1 \dots t_N$ (not the networks' character tokens) are Latin words, existing Khmer text, whitespace and other characters (punctuation, digits, other scripts). The state after unit i is the last Khmer word v in the current sentence ($\langle s \rangle$ at a sentence start), and $V_i(v)$ is the best partial score. Transitions from $V_i(v)$:

- Latin word t_{i+1}: for each $y \in C(t_{i+1})$ with words $u_1 \dots u_k$, $V_{i+1}(u_k) \leftarrow V_i(v) + \alpha\, s(y) + \log P(u_1 \mid v) + \sum_{j>1} \log P(u_j \mid u_{j-1})$, where $s(y) = \log p_\theta(y \mid x) + \gamma \log q_\phi(x \mid y)$;
- joined span (when t_{i+2} is a single space and t_{i+3} a Latin word): the same with the concatenated letters, landing at position $i + 3$, minus β;
- existing Khmer text: its vocabulary words are scored by the bigram model and the state moves to its last word;
- whitespace: no change;
- any other character: the sentence ends, adding $\log P(\langle /s \rangle \mid v)$, and the state resets to $\langle s \rangle$, keeping only the best path.

The final score adds $\log P(\langle /s \rangle \mid v)$. Candidates are the 8-best of each span; an empty candidate is used only if a span has no other.

B Model variants

Two-direction model. One network with the shared 109-entry character table was trained in both directions, with the forward recipe and data plus a reverse batch on 70% of the steps. Over three seeds its forward direction was not better than the forward-only model: +0.1 points on the fresh test [−2.9, +3.0] and +0.28 points of Messages A CER [−0.26, +0.84]. Used as a reverse scorer of its own candidates, its reverse direction lowered the CER by −0.51 [−0.94, −0.14].

Reverse scorers. The first reverse-score comparison (Messages B) used the reverse direction of this two-direction model with the forward-only word model. A reverse-only model of the same architecture, trained only on reverse pairs and selected by held-out negative log-likelihood, was compared on the tuning half and was no better (+0.07 points [−0.10, +0.25]), so the shared model was kept for that decision. For the release, both networks were retrained without the provider-specific rows, the reverse one as a reverse-only model (shown on the tuning half to be about as good). The released package contains the forward-only word model and this reverse-only model.

C Evaluation design and baseline exposure

Exposure of the baseline. On the first word test the three-LLM model scored −8.3 points relative to the baseline, but the baseline had trained on 130 of its 160 words while target exclusion kept them all out of our models. On the 30 words the baseline had not seen, our model scored 65.0% against 60.0%, and on the fresh test (pair exclusion for every system; 73 of its words were training targets of the baseline) the ranking reversed: 48.5% against 34.2%, +14.3 points [+7.2, +21.3]. The first test measured target exposure as much as spelling.

A filter that removed the commonest words. The first blinded comparison, on messages, preferred the baseline (21 against 8). Most losses were common chat words, such as *nis* written as និស instead of នេះ: target exclusion had removed every training row of the first test's words, which were the commonest chat words. After retiring that test into training and moving to pair exclusion, these errors largely disappeared from the messages, while the fresh-test score moved by −3.3 points [−9.2, +2.7]. A held-out filter has to be judged by what it removes as well as what it protects, and the change of policy also changed what the word tests measure.

D Candidate coverage

As an oracle bound on the 1103 Latin words of the development messages, 76.9% can be written right with the top candidate, 96.7% with some candidate among 8 and 97.8% among 32; the contextual decoding wrote 92.7%. On these candidate sets and messages, about 4 points of words are lost in the choice and about 3 are absent from the candidates, and wider beams add little.

E Retraining for the release

The released models were retrained without the LLM spellings that only one provider had given, and without drafts of unknown origin. Over three seed pairs, fresh-word accuracy changed by −1.8 points [−5.2, +1.4], the public romanised test by +2.2 [+0.8, +3.5], and the Messages B CER by +1.10 [+0.82, +1.39], with per-seed differences +1.18 / +2.91 / −0.79. Messages B were written by the provider whose spellings were removed, which may tilt that comparison towards the model that kept them. The blinded comparison set the released pipeline against the previously adopted one (an earlier forward-only word model trained with the rows, with the two-direction model as reverse scorer): among messages where they differed, the judge preferred the previous pipeline in 15 cases and the released one in 10 (5 the same; one-sided $p = 0.21$ for the previous pipeline). The three-seed comparison above paired reverse-only scorers trained with and without the rows.

The released system's development figures are 8.95% CER and 36.9% exact messages on all 450 Messages B (8.93% on the 165 former test messages). These messages chose γ and the released seed, so the figures are optimistic. Its fresh-test accuracy is 58.2% (macro; 60.3% of spellings): 55.8% on the 70 words seen in training with other spellings and 60.3% on the 78 words absent from it.

F Learning curves and retraining variation

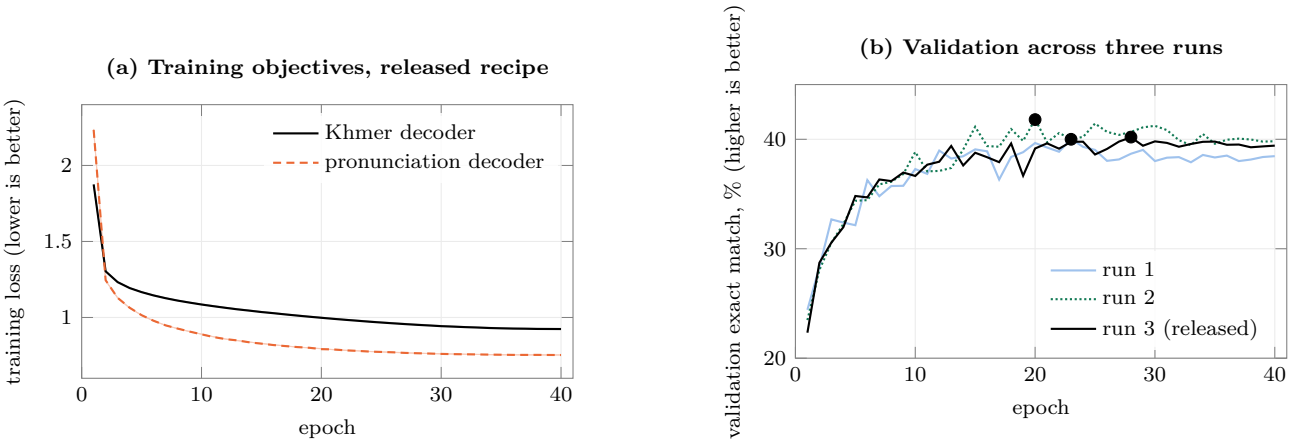


Figure 4: Learning curves of the released word-model recipe, three runs with different seeds, one point per epoch (36,560 updates in 40 epochs). (a) Training loss: label-smoothed cross-entropy of the Khmer decoder, averaged over each epoch's word pairs, and of the training-only pronunciation decoder, averaged over that epoch's side-task steps (every 8th update); the released run is drawn dark, the other two light. (b) Greedy exact match, averaged over the three validation word sets. Dots mark the kept checkpoint of each run: the epoch with the best validation score (epochs 23, 20 and 28).

Figure 4 shows how the released recipe trains. Both training losses fall quickly over the first few epochs and then flatten; with label smoothing they cannot reach zero. Validation exact match peaks between epochs 20 and 28 (best 0.400, 0.418, 0.402) and is lower at epoch 40 (0.385, 0.398, 0.394) while the training loss keeps falling, a mild sign of fitting the training spellings at the expense of held-out ones; selecting the best validation epoch avoids the late decline. The three runs follow the same course but differ by up to about two points at most epochs. In the earlier checkpoint-selection study, the message error rate on development messages kept changing

by up to about one point between neighbouring epochs late in training (Figure 5b), and choosing epochs on a small message set was no better than choosing them on validation words.

Retraining the same recipe with three seeds moved the fresh-test score with a standard deviation of 2.4–3.0 points (ranges up to 5.8) and the Messages A CER by 0.6–1.0 points (ranges up to 1.9). A data-cleaning change that looked harmful on one seed (CER 3.49% → 4.84%) gave per-seed differences of +1.35 / −1.48 / +0.81 points over three. In these experiments, single-seed gains of a few points were therefore unreliable. Early validation scores also ranked recipes poorly: the rank correlation between validation accuracy at epoch 10 and the final fresh-test score was as low as −0.66 over six models, so short runs did not screen recipes in this setting.

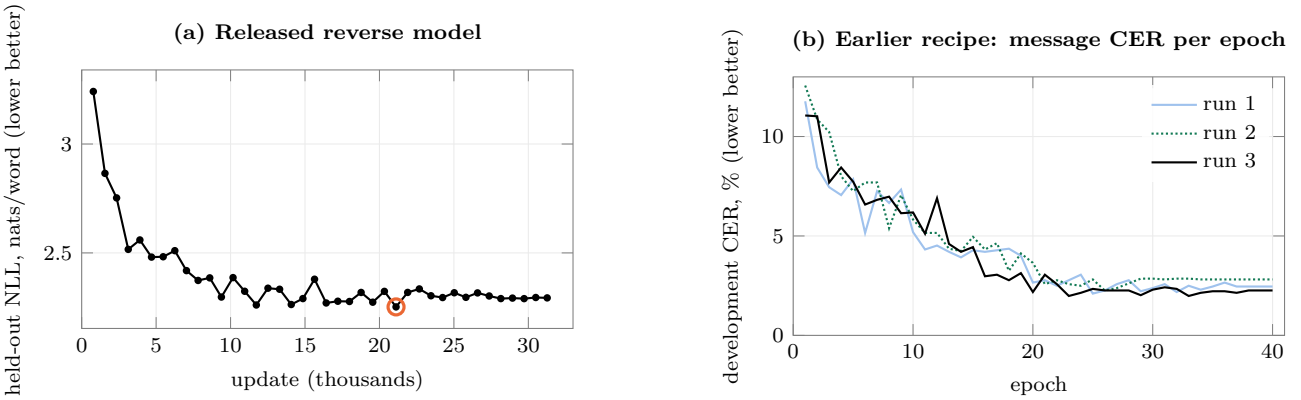


Figure 5: (a) The released reverse model, evaluated every 782 updates and at the last update (31,265): negative log-likelihood of the real spellings of held-out Khmer words, the checkpoint criterion; the circle marks the kept update (21,114). (b) The recipe before the provider-specific rows were removed, three runs: character error rate of contextual decoding on 141 development messages (half of the Messages A fit split), measured at every epoch for the checkpoint-selection comparison. These messages are development data; their other half judged the selection rules (Appendix G). The best-validation epochs of these runs were 37, 23 and 22.

G Intermediate comparisons

Training mixtures. Formal sources did not teach chat conventions: with the public dictionary-style pairs, the rule-romanised lexicon and the annotator's training words, the model scored −21.7 points relative to the baseline on the first word test. Removing the rule-romanised rows, then 108k of about 182k pairs per epoch, changed that test by only +2.6 points. Spellings from three LLMs for 2.6 times more words added a further +4.2 points over the one-LLM model (one seed for the three-LLM model). Table 6 gives the series.

Question	Comparison	Result
Did LLM spellings help the word model?	+ one LLM's filtered spellings vs without (3 seeds, first word test)	+6.5 points [+3.1, +10.1]
Did baseline exposure distort the test?	three-LLM model vs baseline: first test (baseline trained on 130 of 160 words) vs fresh test	−8.3 vs +14.3 points [+7.2, +21.3]
Does sentence context help?	word by word vs contextual decoding, Messages A fit, CER	11.4% → 4.4%; blinded: system with contextual decoding preferred 45, baseline (word by word) preferred 3, same 0, both bad 4
Does the reverse score help?	$\gamma = 1$ vs $\gamma = 0$, Messages B test, CER	10.56% → 9.37% (−1.19 [−1.84, −0.51]); blinded: with reverse score preferred 17, without preferred 7, same 5, both bad 11
What did removing provider-specific rows cost?	without vs with, 3 seed pairs	fresh −1.8 [−5.2, +1.4]; Messages B CER +1.10 [+0.82, +1.39]; blinded: released system preferred 10, previous pipeline preferred 15, same 5, both bad 0
How much does retraining move results?	same recipe, 3 seeds	fresh-test SD 2.4–3.0 points; CER SD 0.6–1.0 points

Table 5: Summary of the comparisons in this paper. Differences are points (percentage points of accuracy or CER); brackets are 95% intervals. Blinded counts are among messages sampled from those where the two systems wrote different Khmer, judged by one annotator.

Training data (word model alone)	First test	Fresh test
Without public pairs or new data	24.0	–
All sources, incl. rule-romanised rows	41.4	–
Without rule-romanised rows	44.0	–
+ one LLM's spellings	50.6	44.8
+ three LLMs' spellings*	54.8	48.5
+ common words unblocked*	–	45.2
Baseline (public pairs + web-mined)	63.1	34.2

Table 6: Word accuracy (macro over words) as the historical training pool changed; three seeds averaged except * (one seed). The first test used target exclusion for our models, not for the baseline.

- **Formal sources.** Adding the public pairs, the rule-romanised lexicon and the annotator's words to a model without them raised the first test by +17.4 points; the dictionary-style public test moved from 64.3% to 67.8% when the rule rows were removed (baseline 70.0%).
- **Three LLMs.** Three-LLM spellings against one: +4.2 points [−1.3, +9.9] on the first test; public names test 48.4% against the baseline's 48.0%.
- **Fresh test, variants and near misses.** On other writers' sound-alike spellings the three-LLM model scored 34.2% against the baseline's 24.1% (+10.1); every system lost 10–14 points there. 39 of 179 outputs were one Khmer character from the right word (baseline 42).
- **First blinded comparison** (both word by word): three-LLM word model preferred 8, baseline preferred 21, same 2, both bad 9, of 40 messages drawn from those where the two differed.
- **Second blinded comparison** (word model with contextual decoding vs baseline, word by word): 52 messages, 40 romanised and 12 mixed with Khmer; one message also appeared in the first comparison.
- **Corrections to contextual decoding.** Normalising the bigram probabilities and allowing joins lowered the Messages A check CER from 3.87% to 3.49%; joins changed 22 messages.
- **Cleaning wrong web-mined pairs.** The web-mined pairs held 284 rows for the 1,000 commonest words, about half wrong in a sample. Removing 531 of them lowered how often the wrong spelling stayed among a word's 8 candidates (73.0% → 58.0%); CER differences over three seeds were +1.35 / −1.48 / +0.81 and fresh-test differences +0.6 / −3.4 / −0.4.
- **Checkpoint selection.** Mean Messages A CER over three seeds under five selection rules: best validation accuracy 1.84%, last epoch 1.93%, average of three epochs 1.90%, best development-message CER 2.06%, best recall at 8 2.96%.
- **Web text and chat.** In 20,000 Khmer web documents, 16.7% of word occurrences were among the 100 commonest chat words, against 59.8% in chat-style messages.
- **Third and fourth blinded comparisons.** 40 of the 75 Messages B test messages on which the reverse score changed the output; 30 of the 169 Messages B messages on which the release models differed from their predecessors. Four messages appear in both.